

Java Classes

Java has one keyword for declaring a class, and many shapes a class can take. Some are language features, some are patterns we write by hand. This covers the ones a working developer meets, most important first.

Ranked by how often they appear in real code and in interviews:

- | | | |
|-------------|------------------|-------------|
| 1. Concrete | 6. Static nested | 11. Utility |
| 2. Abstract | 7. Inner | 12. Generic |
| 3. POJO | 8. Anonymous | 13. Local |
| 4. Record | 9. Singleton | |
| 5. Enum | 10. Final | |

1. Concrete class

A concrete class is the ordinary one. Every method has a body, so we can create objects from it. This is what we write unless there is a reason not to.

```
class Car {
    private String brand;
    private int speed;

    Car(String brand, int speed) {
        this.brand = brand;
        this.speed = speed;
    }

    void drive() {
        System.out.println(brand + " is driving at " + speed);
    }
}

public class Main {
    public static void main(String[] args) {
        Car car = new Car("Toyota", 100);
        car.drive();
    }
}
```

Output:

```
Toyota is driving at 100
```

2. Abstract class

An abstract class cannot be created directly. It holds the behavior its subclasses share, and declares the methods each one must supply.

```
abstract class Shape {
    abstract double area();

    void describe() {
        System.out.println("Area is " + area());
    }
}

class Circle extends Shape {
    private final double radius;

    Circle(double radius) {
        this.radius = radius;
    }

    @Override
    double area() {
        return 3.14 * radius * radius;
    }
}

public class Main {
    public static void main(String[] args) {
        Shape shape = new Circle(2);
        shape.describe();
    }
}
```

Output:

```
Area is 12.56
```

`new Shape()` does not compile. An abstract class exists to be extended.

3. POJO class

POJO means Plain Old Java Object. It holds data and nothing else: private fields, a no-argument constructor, getters and setters, and no dependency on any framework.

```

class Employee {
    private String name;
    private double salary;

    public Employee() { }

    public Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public double getSalary() { return salary; }
    public void setSalary(double salary) { this.salary = salary; }
}

```

Using it:

```

public class Main {
    public static void main(String[] args) {
        Employee employee = new Employee("Ravi", 50000);
        employee.setSalary(55000);

        System.out.println(employee.getName() + " earns " + employee.getSalary());
    }
}

```

Output:

```
Ravi earns 55000.0
```

POJOs are what most frameworks expect: request bodies, database rows, and messages are usually POJOs.

4. Record class

A record is the short way to write a POJO that never changes. Java generates the constructor, the accessors, `equals`, `hashCode` and `toString`. It arrived in Java 16.

```

record Employee(String name, double salary) { }

public class Main {
    public static void main(String[] args) {
        Employee employee = new Employee("Ravi", 50000);

        System.out.println(employee.name());
        System.out.println(employee);
        System.out.println(employee.equals(new Employee("Ravi", 50000)));
    }
}

```

Output:

```

Ravi
Employee[name=Ravi, salary=50000.0]
true

```

Thirty lines of POJO became one. The accessor is `name()`, not `getName()`, and there is no setter, because a record cannot change after it is created.

```

POJO    → data that changes, setters included
Record → data that never changes

```

5. Enum class

An enum is a class with a fixed set of instances. Each constant is an object, so an enum can carry fields and methods like any other class.

```

enum Status {
    ACTIVE("Working"),
    INACTIVE("Not working"),
    PENDING("Waiting for approval");

    private final String description;

    Status(String description) {
        this.description = description;
    }

    String getDescription() {
        return description;
    }
}

public class Main {
    public static void main(String[] args) {
        Status status = Status.PENDING;

        System.out.println(status);
        System.out.println(status.getDescription());

        for (Status value : Status.values()) {
            System.out.println(value + " = " + value.getDescription());
        }
    }
}

```

Output:

```

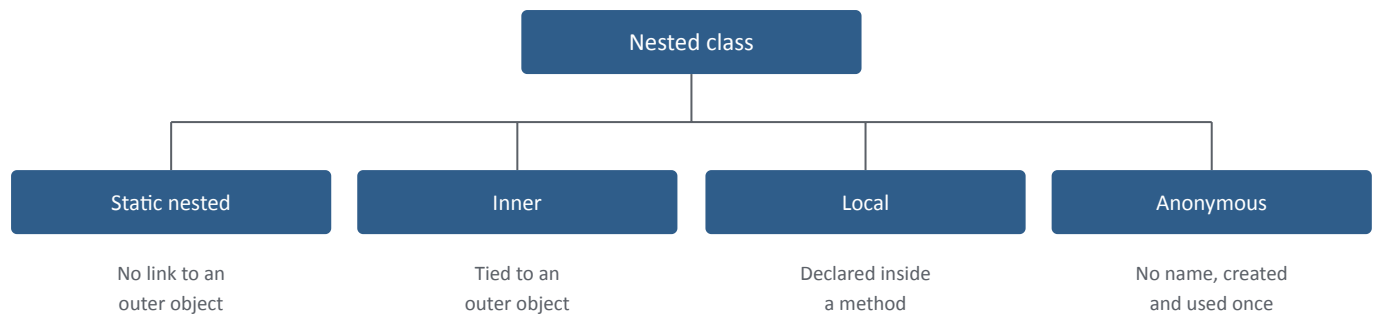
PENDING
Waiting for approval
ACTIVE = Working
INACTIVE = Not working
PENDING = Waiting for approval

```

An enum is better than a set of `String` constants, because the compiler rejects a value that is not in the list.

Nested classes

Four of the next sections are kinds of nested class, which is any class declared inside another class.



Only the first two are declared as members of the outer class. The other two are declared inside a method.

6. Static nested class

A static nested class sits inside another class but has no link to an outer object. We create it without creating the outer class first.

The common use is a builder.

```

class Pizza {
    private final String size;
    private final boolean cheese;

    private Pizza(Builder builder) {
        this.size = builder.size;
        this.cheese = builder.cheese;
    }

    static class Builder {
        private String size = "medium";
        private boolean cheese = false;

        Builder size(String size) {
            this.size = size;
            return this;
        }

        Builder cheese(boolean cheese) {
            this.cheese = cheese;
            return this;
        }

        Pizza build() {
            return new Pizza(this);
        }
    }

    @Override
    public String toString() {
        return size + " pizza, cheese: " + cheese;
    }
}

```

Using it:

```

public class Main {
    public static void main(String[] args) {
        Pizza pizza = new Pizza.Builder()
            .size("large")
            .cheese(true)
            .build();

        System.out.println(pizza);
    }
}

```

Output:

```
large pizza, cheese: true
```

`new Pizza.Builder()` needs no `Pizza` object. That is what `static` gives us here.

7. Inner class

An inner class is a nested class without `static`. Every instance belongs to an instance of the outer class, so it can read the outer object's private fields.

```
class Car {
    private String brand = "Toyota";

    class Engine {
        void start() {
            System.out.println("Starting the engine of " + brand);
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Car car = new Car();
        Car.Engine engine = car.new Engine();

        engine.start();
    }
}
```

Output:

```
Starting the engine of Toyota
```

The `car.new Engine()` syntax shows why: an inner class cannot exist without an outer object.

	Static nested	Inner
Declared	<code>static class</code>	<code>class</code>
Created with	<code>new Outer.Nested()</code>	<code>outer.new Inner()</code>
Outer object	Not needed	Required
Outer private fields	Not reachable	Reachable
Memory	Independent	Keeps the outer object in memory

Prefer static nested. An inner class keeps a reference to the outer object, which is a common cause of memory leaks.

8. Anonymous class

An anonymous class has no name. We declare it and create the one instance of it in the same expression.

```
interface Greeting {
    void greet();
}

public class Main {
    public static void main(String[] args) {
        Greeting formal = new Greeting() {
            @Override
            public void greet() {
                System.out.println("Good morning");
            }
        };

        // Since Java 8, a lambda does the same job in one line
        Greeting casual = () -> System.out.println("Hey");

        formal.greet();
        casual.greet();
    }
}
```

Output:

```
Good morning
Hey
```

A lambda replaces an anonymous class only when the interface has exactly one method. For anything else, or when we need state, the anonymous class is still the right choice.

9. Singleton class

A singleton allows exactly one instance. The constructor is private, and a static method returns the single object.

```

class Settings {
    private static final Settings INSTANCE = new Settings();

    private String theme = "dark";

    private Settings() { }

    static Settings getInstance() {
        return INSTANCE;
    }

    String getTheme() {
        return theme;
    }
}

public class Main {
    public static void main(String[] args) {
        Settings a = Settings.getInstance();
        Settings b = Settings.getInstance();

        System.out.println(a.getTheme());
        System.out.println(a == b);
    }
}

```

Output:

```

dark
true

```

`a == b` is true because both names point at the same object.

An enum gives the same guarantee with less code, and it is safe across threads and serialization without any extra work:

```

enum Settings {
    INSTANCE;

    private String theme = "dark";

    String getTheme() {
        return theme;
    }
}

```

10. Final class

A final class cannot be extended. We mark a class final when subclassing would break something it promises.

```
final class Money {
    private final double amount;

    Money(double amount) {
        this.amount = amount;
    }

    double getAmount() {
        return amount;
    }
}

// class Rupee extends Money { }    // Error: cannot inherit from final Money
```

`String`, `Integer` and `LocalDate` are all final. `String` has to be, because the whole language relies on a string never changing after it is created.

11. Utility class

A utility class holds only static methods. It is never created, so the constructor is private and the class is final.

```
final class MathUtils {

    private MathUtils() { }    // nobody can create one

    static int square(int n) {
        return n * n;
    }

    static int max(int a, int b) {
        return a > b ? a : b;
    }
}

public class Main {
    public static void main(String[] args) {
        System.out.println(MathUtils.square(5));
        System.out.println(MathUtils.max(3, 9));
    }
}
```

Output:

25
9

`Math`, `Collections` and `Arrays` in the JDK are all built this way.

12. Generic class

A generic class works with many types while the compiler still checks them. The type is supplied when the object is created.

```
class Box<T> {
    private T item;

    void put(T item) {
        this.item = item;
    }

    T get() {
        return item;
    }
}

public class Main {
    public static void main(String[] args) {
        Box<String> stringBox = new Box<>();
        stringBox.put("Hello");
        System.out.println(stringBox.get());

        Box<Integer> intBox = new Box<>();
        intBox.put(42);
        System.out.println(intBox.get() + 1);
    }
}
```

Output:

Hello
43

Without generics the field would be `Object`, and every read would need a cast that could fail at run time.

13. Local class

A local class is declared inside a method and exists only there. It is the rarest of the four nested kinds, and a lambda or a static nested class usually reads better.

```

public class Main {

    static void process(String prefix) {
        class Labeller {
            String label(String text) {
                return prefix + ": " + text;
            }
        }

        Labeller labeller = new Labeller();
        System.out.println(labeller.label("order created"));
    }

    public static void main(String[] args) {
        process("INFO");
    }
}

```

Output:

```
INFO: order created
```

A local class can read the method's variables, as long as they never change after being set.

The types together

Rank	Type	What it is	When we use it
1	Concrete	An ordinary class	The default
2	Abstract	Cannot be created, holds shared behavior	Several subclasses share code
3	POJO	Private fields with getters and setters	Data that changes
4	Record	An immutable data carrier	Data that does not change
5	Enum	A fixed set of instances	A closed list of values
6	Static nested	Inside a class, no outer link	Builders, helpers
7	Inner	Inside a class, tied to an instance	Needs the outer object's state
8	Anonymous	No name, created once	A one-off implementation
9	Singleton	Exactly one instance	Shared configuration or a cache
10	Final	Cannot be extended	Subclassing would break a promise
11	Utility	Only static methods	A group of stateless helpers
12	Generic	Works with many types	A container or a reusable algorithm

Rank	Type	What it is	When we use it
13	Local	Declared inside a method	Rare. Prefer a lambda

For example, imagine an order service. `Order` and `Customer` are records, because they never change once created. `OrderStatus` is an enum. `OrderProcessor` is a concrete class. Its `Builder` is a static nested class. `PriceCalculator` is a utility class of static methods. The database configuration is a singleton, and the cache is a generic class.

The most useful thing at this point is not to memorize the list. Open a class from a real project and name which of the thirteen it is, and whether a different one would suit it better.